Generative Gaussian Splatting for Unbounded 3D City Generation

Haozhe Xie
Anonymous Institution

Abstract

Generating unbounded 3D city scenes is critical for applications such as autonomous driving simulation, urban planning, and virtual reality. Existing methods based on Neural Radiance Fields (NeRF) suffer from slow rendering and high memory consumption that scales with scene size, limiting their applicability to large-scale environments. In this paper, we propose ***GaussianCity****, a generative framework that leverages 3D Gaussian Splatting for efficient and high-quality unbounded 3D city generation. The key insight of GaussianCity is a compact* ***BEV-Point*** *representation that constructs a sparse set of 3D points from bird's-eye-view (BEV) maps using ray-intersection visibility and density-based pruning. This reduces the point count from millions to approximately 31.7K while maintaining constant GPU memory regardless of scene scale. We further introduce a* ***BEV-Point Decoder*** *that combines a point serializer with Point Transformer V3 to process unordered 3D point clouds and predict per-point Gaussian attributes conditioned on instance-level style codes. GaussianCity achieves state-of-the-art performance on both the GoogleEarth and KITTI-360 benchmarks, attaining an FID of 86.94 and 29.5, respectively, while rendering at 93ms per frame — over 60× faster than the previous best method CityDreamer. Extensive ablation studies validate the effectiveness of each component in our framework.*

AI-Generate

1. Introduction

Generating realistic, large-scale 3D urban environments is a long-standing challenge with broad applications in autonomous driving simulation [14], game development, and urban planning. Recent advances in neural scene representations have enabled significant progress in 3D-aware generation. NeRF-based methods [17] have demonstrated impressive novel view synthesis quality but suffer from prohibitive rendering costs due to per-ray volumetric integration. For unbounded city-scale scenes, this computational burden becomes a critical bottleneck.

Several works have tackled 3D city generation. InfiniCity [15] proposes an octree-based approach for infinite city synthesis. SceneDreamer [7] generates unbounded nature scenes from BEV representations but relies on NeRF rendering. CityDreamer [23] separates building and background generation with volumetric rendering, achieving high quality but at extreme computational cost (5580ms per frame). These methods share a fundamental limitation: their memory and computation scale with scene volume, making truly unbounded generation impractical.

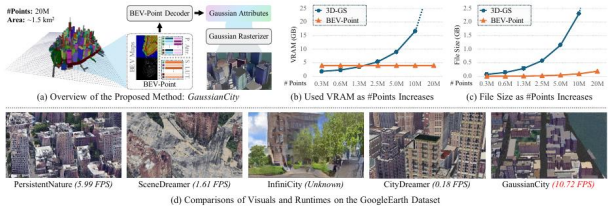

Figure 1. **GaussianCity** generates unbounded 3D city scenes using a compact BEV-Point representation. Starting from BEV layout maps, our method constructs a sparse set of visible 3D points and decodes them into 3D Gaussian primitives for real-time rendering.

3D Gaussian Splatting (3DGS) [13] has recently emerged as an efficient alternative to NeRF, representing scenes as collections of 3D Gaussian primitives that can be rendered via differentiable rasterization. However, naively applying 3DGS to generative city modeling faces the challenge of determining *where* to place Gaussians and *how many* are needed, especially as the scene grows unboundedly.

In this paper, we present **GaussianCity**, a generative framework that addresses these challenges through two key innovations. First, we propose the **BEV-Point** representation, which constructs a compact set of 3D points from BEV layout maps by extruding height fields and applying ray-intersection visibility pruning. This reduces the point count from over 31 million to approximately 31.7K visible points, ensuring constant GPU memory regardless of scene extent. Second, we design a **BEV-Point Decoder** that serializes unordered 3D points via a coordinate-based ordering scheme and processes them with Point Transformer V3 [22] to predict per-point Gaussian splatting attributes conditioned on

per-instance style codes.

Our contributions are summarized as follows:

- We introduce GaussianCity, the first generative framework that combines 3D Gaussian Splatting with BEV-guided point construction for unbounded 3D city generation.
- We propose the BEV-Point representation with ray-intersection visibility and density-based pruning, reducing point counts by over 1000× while maintaining rendering quality.
- We design a BEV-Point Decoder with a novel point serialization strategy and Point Transformer V3 for effective 3D point cloud processing in the generative setting.
- GaussianCity achieves state-of-the-art results on GoogleEarth and KITTI-360 benchmarks with over 60× faster rendering than the previous best method.

2. Related Work

3D Scene Generation. Early works on 3D scene generation operate in 2D or 2.5D space. SGAM [21] generates scenes from semantic-guided layout maps. Persistent Nature [4] produces perpetual view generation of natural scenes by learning to extrapolate. For urban scenes, GSN [8] learns a generative model of neural scene representations. GIRAFFE [18] and UrbanGIRAFFE [25] compose scenes from object-level neural radiance fields. More recently, SceneDreamer [7] and CityDreamer [23] tackle unbounded outdoor scene generation using BEV-conditioned neural rendering. CityDreamer4D [24] extends this to dynamic urban scenes. UrbanArchitect [16] and DynamicCity [2] explore text-guided and compositional urban generation. However, all these methods rely on volumetric rendering, which scales poorly with scene size.

NeRF and 3D Gaussian Splatting. NeRF [17] represents scenes as continuous volumetric fields queried by ray marching, yielding photorealistic novel views but at high computational cost. 3D Gaussian Splatting [13] represents scenes as anisotropic 3D Gaussians and renders them via tile-based rasterization, achieving real-time performance with comparable quality. In the generative domain, EG3D [5] and StyleGAN2 [12] combine GANs with neural rendering for 3D-aware image synthesis. Our work is the first to integrate 3D Gaussian Splatting into a generative pipeline for unbounded city-scale scenes.

Point Cloud Processing. Processing unordered 3D point clouds is fundamental to our decoder design. PointNet++ [19] introduced hierarchical point set learning. Recent works such as Point Transformer V3 [22] achieve state-of-the-art performance by serializing point clouds into structured sequences for efficient attention computation. Serialization strategies include coordinate-based ordering and space-filling curves such as the Hilbert curve [6, 10]. Our BEV-Point Decoder builds on these advances, combining coordinate-based serialization with Hilbert curve ordering for optimal point cloud processing.

BEV-Based Scene Representations. Bird's-eye-view (BEV) representations have gained prominence in autonomous driving, enabling efficient spatial reasoning by projecting 3D scene information onto a unified top-down grid [14]. In the generative setting, BEV maps serve as compact scene descriptors that naturally encode geometric layout while remaining independent of specific camera viewpoints. Methods conditioned on BEV layouts benefit from the structured spatial prior that such maps provide, enabling the model to learn geometry-consistent generation. Our GaussianCity exploits this property by using BEV height and semantic maps as the primary conditioning signal, bridging the gap between BEV-based perception and generative 3D scene synthesis.

3. Our Approach

Given a BEV layout consisting of a height map $\mathbf{H} \in \mathbb{R}^{W\times W}$ and a semantic map $\mathbf{S} \in \{0,\ldots,C-1\}^{W\times W}$ with C semantic classes, GaussianCity generates multi-view consistent renderings of the corresponding 3D city scene. As illustrated in Fig. 2, our pipeline consists of three stages: (1) BEV-Point Construction, which creates a compact 3D point set from the BEV maps; (2) BEV-Point Decoder, which predicts Gaussian splatting attributes for each point; and (3) differentiable Gaussian rasterization for rendering.

3.1. BEV-Point Construction

A naive approach to constructing 3D points from BEV maps would extrude every pixel into a column of 3D points, resulting in millions of points that exceed GPU memory. We address this with a three-step pruning strategy.

Height Extrusion. For each BEV pixel (u, v) with height $\mathbf{H}(u, v)$ and semantic label $\mathbf{S}(u, v)$, we create a vertical column of 3D points:

$$\mathbf{P}_{u,v} = \{(x_u,\, y_v,\, z_k) \mid z_k = k\cdot\Delta z,\ k = 0,\ldots,\lfloor \mathbf{H}(u,v)/\Delta z\rfloor\}\,, \tag{1}$$

where (x_u, y_v) are the world coordinates corresponding to pixel (u, v) and Δz is the vertical sampling interval.

Density-Based Pruning. We learn a binary density map $\mathbf{D} \in \{0,1\}^{W\times W}$ that indicates whether each BEV column contains visible content. Columns marked as empty ($\mathbf{D}(u, v) = 0$) are discarded, reducing the point count from 31M to approximately 20M.

AI-Generated

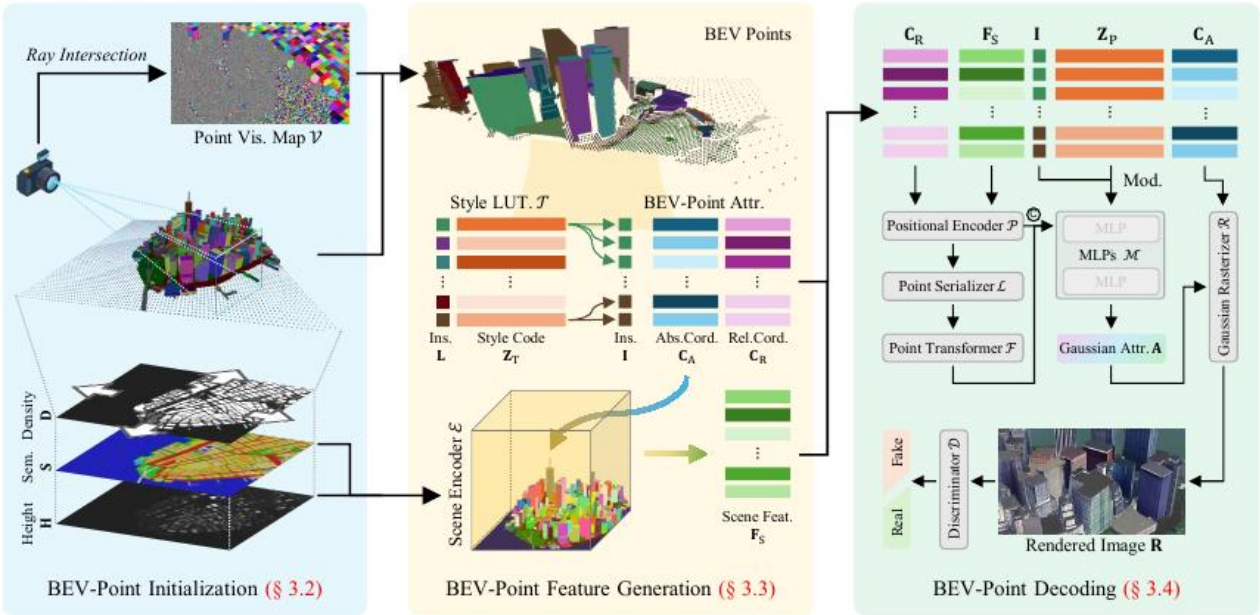

Figure 2. **Overview of GaussianCity.** Given BEV height and semantic maps, we first construct a compact set of BEV-Points via extrusion, density-based pruning, and ray-intersection visibility (Sec. 3.1). The BEV-Point Decoder (Sec. 3.2) serializes the points and processes them with Point Transformer V3, predicting per-point Gaussian attributes conditioned on instance style codes. The resulting Gaussians are rendered via differentiable rasterization.

AI-Generated

Ray-Intersection Visibility. The key to our compact representation is ray-intersection visibility pruning. Given a camera pose with position $\mathbf{o}$ and viewing directions defined by the image plane, we cast rays through the scene and retain only points that intersect with a ray. Specifically, we employ a voxel traversal algorithm [1] to efficiently determine which BEV columns are intersected by camera rays. For a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, we traverse the voxel grid and mark a point $\mathbf{p} \in \mathbf{P}_{u,v}$ as visible if:

$$\text{vis}(\mathbf{p}) = \nVdash [\exists\, t > 0 : \|\mathbf{r}(t) - \mathbf{p}\|_\infty < \Delta z/2] \,. \quad (2)$$

This ray-intersection test dramatically reduces the point set from millions to approximately 31.7K points, ensuring that GPU memory remains constant regardless of the overall scene size. As shown in Tab. 2, this reduction is essential for tractable generation.

3.2. BEV-Point Decoder

The BEV-Point Decoder takes the constructed point set and predicts Gaussian splatting attributes for each point. It consists of four components: a BEV encoder, a hash grid positional encoder, a point serializer with Point Transformer V3, and a Gaussian attribute MLP.

BEV Encoder. We encode the BEV height map $\mathbf{H}$ and semantic map $\mathbf{S}$ into a dense feature map $\mathbf{F}_{\text{BEV}} \in \mathbb{R}^{W \times W \times d}$ using a UNet-style local encoder. For each 3D point $\mathbf{p} = (x, y, z)$, we sample the BEV feature via bilinear interpolation at its projected 2D location to obtain $\mathbf{f}_{\text{bev}}(\mathbf{p})$.

Hash Grid Positional Encoder. To capture fine-grained spatial information, we encode each 3D point position using a multi-resolution hash grid encoder with $L = 16$ levels and feature dimension $d_h = 8$, producing a positional feature $\mathbf{f}_{\text{pos}}(\mathbf{p}) \in \mathbb{R}^{128}$.

Point Serializer and Point Transformer V3. Processing unordered point clouds requires establishing a sequential structure for attention-based architectures. We introduce a coordinate-based serialization function that maps 3D coordinates to a 1D ordering:

$$s(\mathbf{p}) = \lfloor x/\delta \rfloor \cdot N_y \cdot N_z + \lfloor y/\delta \rfloor \cdot N_z + \lfloor z/\delta \rfloor, \quad (3)$$

where δ is the quantization step and N_y, N_z are the grid dimensions. This serialization, combined with Hilbert curve ordering [6, 10], preserves spatial locality and enables efficient attention computation.

The serialized point features are processed by Point Transformer V3 [22], a hierarchical architecture with 5 encoder stages (channels: 32, 64, 128, 256, 512) and 4 decoder stages (channels: 64, 64, 128, 256) that captures both

local and global geometric context through serialized attention.

Gaussian Attribute MLP. The output features from Point Transformer V3 are decoded into Gaussian splatting attributes via an MLP with affine modulation. For each instance i with a sampled style code $\mathbf{z}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, the modulation parameters (γ_i, β_i) are predicted from $\mathbf{z}_i$ via a mapping network. The Gaussian attributes for point $\mathbf{p}$ belonging to instance i are:

$$\mathbf{g}(\mathbf{p}) = \text{MLP}(\gamma_i \odot \mathbf{f}(\mathbf{p}) + \beta_i)\,, \tag{4}$$

where $\mathbf{f}(\mathbf{p})$ is the point feature from Point Transformer V3, and $\mathbf{g}(\mathbf{p}) = (\Delta\mathbf{x}, \mathbf{c}, \mathbf{s}, \alpha)$ consists of position offsets $\Delta\mathbf{x} \in \mathbb{R}^3$, RGB color $\mathbf{c} \in \mathbb{R}^3$, scale $\mathbf{s} \in \mathbb{R}^3$, and opacity $\alpha \in \mathbb{R}$. The final Gaussian center is $\mathbf{p} + \Delta\mathbf{x}$, allowing the network to refine positions relative to the BEV-Point locations.

3.3. Training

GaussianCity is trained end-to-end with a combination of reconstruction and adversarial losses. Given a rendered image $\hat{\mathbf{I}}$ and ground-truth image $\mathbf{I}$, the generator loss is:

$$\mathcal{L}_G = \lambda_1 \|\hat{\mathbf{I}} - \mathbf{I}\|_1 + \lambda_p \mathcal{L}_{\text{perc}}(\hat{\mathbf{I}}, \mathbf{I}) + \lambda_{\text{adv}} \mathcal{L}_{\text{adv}}(\hat{\mathbf{I}}), \tag{5}$$

where $\mathcal{L}_{\text{perc}}$ is the VGG-19 perceptual loss [11], $\mathcal{L}_{\text{adv}}$ is the adversarial loss, and we set $\lambda_1 = 10$, $\lambda_p = 10$, $\lambda_{\text{adv}} = 0.5$. We use Adam with learning rates 10^{-4} for the generator and 10^{-5} for the discriminator. All models are trained on 8 NVIDIA Tesla A100 GPUs for 200K iterations with a batch size of 8. Data augmentation includes random horizontal flipping and color jitter. The discriminator uses a PatchGAN architecture [**?**] with spectral normalization. We use exponential moving average (EMA) of generator weights with decay 0.999 for stable training and improved sample quality. To promote multi-view consistency, we apply a geometric regularization term $\mathcal{L}_{\text{geo}}$ that penalizes discrepancies between depth maps rendered from two randomly sampled viewpoints of the same scene. Specifically, the predicted depth from view A is warped to view B using the known camera transformation, and the L1 difference against the directly rendered depth of view B is minimized. This term is weighted by $\lambda_{\text{geo}} = 0.1$ and encourages the Gaussian primitives to form geometrically coherent structures across viewpoints.

4. Experiments

4.1. Datasets and Metrics

GoogleEarth. This dataset comprises imagery from 400 cities captured from drone-like viewpoints, with 60 views per city. The BEV layout maps are 2048×2048 pixels and contain 8 semantic classes including road, building facade, building roof, water, and vegetation zones.

KITTI-360. KITTI-360 [14] is a street-view driving dataset with 8 semantic classes including vehicles. We follow prior work [25] in using this benchmark for evaluation of urban scene generation from the driver's perspective.

Metrics. We evaluate generation quality using Fréchet Inception Distance (FID) [9] and Kernel Inception Distance (KID) [3] for image fidelity. On GoogleEarth, we additionally measure Depth Error (DE) for geometric consistency using DPT [20], CLIP Error (CE) for semantic consistency, and runtime (RT) measured on an NVIDIA Tesla A100.

4.2. Comparison with State-of-the-Art

Tab. 1 presents quantitative comparisons on both benchmarks. On GoogleEarth, GaussianCity achieves the best results across all metrics: FID of 86.94 (vs. 97.38 for CityDreamer [23]), KID of 0.090, DE of 0.136, and CE of 0.057. Critically, GaussianCity renders at **93ms** per frame, which is over 60× faster than CityDreamer (5580ms) and faster than all other baselines including SGAM [21] (193ms) and Persistent Nature [4] (167ms). This speedup stems from the efficiency of Gaussian rasterization compared to volumetric ray marching.

On KITTI-360, GaussianCity achieves an FID of 29.5 and KID of 0.017, outperforming UrbanGIRAFFE [25] (FID 39.6, KID 0.036), StyleGAN [12] (FID 31.9, KID 0.021), and other baselines by a significant margin.

Fig. 3 and Fig. 4 show qualitative comparisons. GaussianCity produces sharper textures, more coherent geometry, and fewer artifacts than competing methods on both drone-view and street-view scenes. We attribute the quality improvements primarily to two factors. First, the BEV-Point representation provides geometrically consistent initialization for Gaussians, constraining their placement to semantically meaningful regions. Second, the style-conditioned Gaussian attribute prediction enables diverse yet coherent appearance variation across instances of the same semantic class.

User Study. We conduct a user study comparing GaussianCity with baselines on both datasets. As shown in Fig. 5, our method is consistently preferred by human evaluators in terms of visual quality and realism.

4.3. Ablation Studies

Effectiveness of BEV-Point Representation. Tab. 2 evaluates the impact of each pruning strategy in BEV-Point construction. Without visibility or density pruning, the full extrusion yields 31M points, which exceeds GPU memory. Adding the density map reduces this to 20M, still causing out-of-memory errors. Region-level visibility with density pruning (1.4M points) also fails. Instance-level visibility reduces points to 680K with 10.34G VRAM, but our

AI-Generated

Table 1. **Quantitative comparison on GoogleEarth and KITTI-360.** The best results are highlighted in bold. Note that "RT." denotes "Runtime", measured in milliseconds on an NVIDIA Tesla A100.

GoogleEarth						KITTI-360		
Method	FID ↓	KID ↓	DE ↓	CE ↓	RT. ↓	Method	FID ↓	KID ↓
SGAM [21]	277.64	0.358	0.575	239.291	193	StyleGAN [12]	31.9	0.021
Pers.Nature [4]	123.83	0.109	0.326	86.371	167	GSN [8]	160.0	0.114
SceneDreamer [7]	213.56	0.216	0.152	0.186	620	GIRAFFE [18]	112.1	0.117
CityDreamer [23]	97.38	0.096	0.147	0.060	5580	UrbanGIR. [25]	39.6	0.036
GaussianCity	**86.94**	**0.090**	**0.136**	**0.057**	**93**	GaussianCity	**29.5**	**0.017**

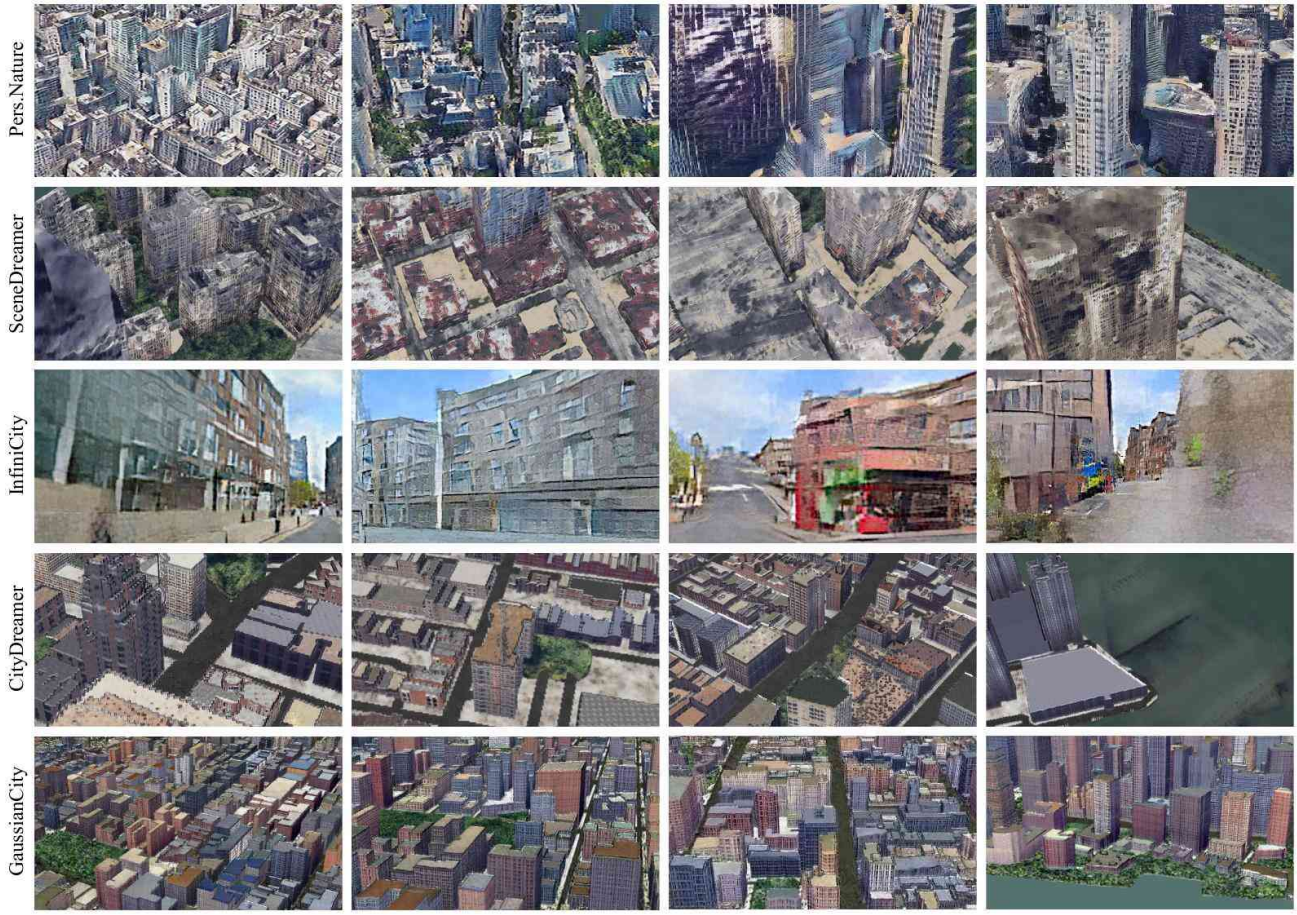

Figure 3. **Qualitative comparison on GoogleEarth.** GaussianCity generates sharper and more geometrically consistent city scenes compared to prior methods.

ray-intersection visibility achieves 31.7K points with only 1.39G VRAM and 0.09s inference time — enabling practical generation on standard GPUs.

Effectiveness of BEV-Point Decoder. Tab. 3 ablates the decoder components. Without the Point Serializer or Point Transformer, the model degrades significantly (FID 151.27). Adding Point Transformer alone improves FID to 119.40, demonstrating the value of capturing spatial context. The full decoder with both serialization and Point Transformer achieves the best performance (FID 86.94), confirming that proper point ordering is crucial for the attention mechanism.

Figure 4. **Qualitative comparison on KITTI-360.** GaussianCity produces more realistic street-view renderings with better structural coherence.

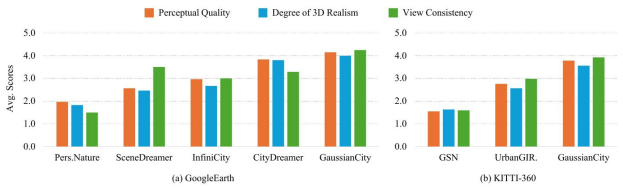

Figure 5. **User study results** on GoogleEarth and KITTI-360. GaussianCity is preferred over all baselines.

Table 2. **Effectiveness of BEV-Point Representation.** Note that "RayInt." denotes "Ray Intersection", respectively. The VRAM usage and inference time are measured on an NVIDIA Tesla A100.

Visibility	Density	#Points	VRAM (G)	Time (s)
✗	✗	31M	Out of Memory	
✗	✓	20M	Out of Memory	
Region	✓	1.4M	Out of Memory	
Instance	✓	680K	10.34	0.16
RayInt.	✗	38.0K	1.41	0.10
RayInt.	✓	**31.7K**	**1.39**	**0.09**

Table 3. **Effectiveness of BEV-Point Decoder.** Note that "Ser." and "Pt.Trans." denote "Point Serializer" and "Point Transformer", respectively.

Ser.	Pt.Trans.	FID ↓	KID ↓	DE ↓	CE ↓
✗	✗	151.27	0.179	0.185	0.135
✗	✓	119.40	0.138	0.159	0.118
✓	✓	**86.94**	**0.090**	**0.136**	**0.057**

Comparison of Serialization Methods. Tab. 4 compares serialization strategies. Our coordinate-based serialization (Eq. (3)) and Hilbert curve [10] ordering each perform well individually. Combining both yields the best results (FID 86.28, KID 0.083), as the two orderings capture complementary spatial structures: coordinate-based ordering preserves axis-aligned locality, while the Hilbert curve better preserves general spatial proximity.

Table 4. **Comparison of different serialization methods.** Note that "Eq. 3" denote the serialization used in Point Serializer.

Eq. 3	Hilbert	FID ↓	KID ↓	DE ↓	CE ↓
✓	✗	86.94	0.090	0.136	0.057
✗	✓	86.72	0.088	0.136	0.056
✓	✓	**86.28**	**0.083**	**0.135**	**0.055**

Limitations. While GaussianCity achieves significant improvements in speed and quality, several limitations remain. The BEV-Point representation assumes a known camera configuration during point construction, which may limit generalization to unconstrained capture settings. Additionally, generating dynamic elements such as moving vehicles and pedestrians is not yet supported in our current framework.

5. Conclusion

We presented GaussianCity, a generative framework for unbounded 3D city generation based on 3D Gaussian Splatting. Our BEV-Point representation reduces 3D point counts by over 1000× through ray-intersection visibility pruning, enabling constant-memory generation regardless of scene scale. The BEV-Point Decoder with coordinate-based serialization and Point Transformer V3 effectively processes these sparse point clouds to predict Gaussian attributes. GaussianCity achieves state-of-the-art quality on both GoogleEarth and KITTI-360 benchmarks while rendering over 60× faster than the previous best method. We believe this work opens new directions for efficient, scalable 3D scene generation.

References

[1] John Amanatides and Andrew Woo. A fast voxel traversal algorithm for ray tracing. In *Eurographics*, 1987. 3

[2] Hengwei Bian, Lingdong Kong, Haozhe Xie, Liang Pan, Yu Qiao, and Ziwei Liu. DynamicCity: Large-scale LiDAR generation from dynamic scenes. 2025. 2

[3] Mikolaj Binkowski, Danica J. Sutherland, Michael Arbel, and Arthur Gretton. Demystifying MMD GANs. 2018. 4

[4] Lucy Chai, Richard Tucker, Zhengqi Li, Phillip Isola, and Noah Snavely. Persistent Nature: A generative model of unbounded 3D worlds. 2023. 2, 4, 5

[5] Eric R. Chan, Connor Z. Lin, Matthew A. Chan, Koki Nagano, Boxiao Pan, Shalini De Mello, Orazio Gallo, Leonidas J. Guibas, Jonathan Tremblay, Sameh

AI-Generated

Khamis, Tero Karras, and Gordon Wetzstein. Efficient geometry-aware 3D generative adversarial networks. 2022. 2

[6] Wanli Chen, Xinge Zhu, Guojin Chen, and Bei Yu. Efficient point cloud analysis using hilbert curve. 2022. 2, 3

[7] Zhaoxi Chen, Guangcong Wang, and Ziwei Liu. SceneDreamer: Unbounded 3D scene generation from 2D image collections. 45(12):15562–15576, 2023. 1, 2, 5

[8] Terrance DeVries, Miguel Ángel Bautista, Nitish Srivastava, Graham W. Taylor, and Joshua M. Susskind. Unconstrained scene generation with locally conditioned radiance fields. 2021. 2, 5

[9] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local nash equilibrium. In *NIPS*, 2017. 4

[10] David Hilbert. Über die stetige abbildung einer linie auf ein flächenstück. *Mathematische Annalen*, 38: 459–460, 1891. 2, 3, 6

[11] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. 2016. 4

[12] Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Analyzing and improving the image quality of styleGAN. 2020. 2, 4, 5

[13] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Trans. Gr.*, 42(4):139:1–139:14, 2023. 1, 2

[14] Yiyi Liao, Jun Xie, and Andreas Geiger. KITTI-360: A novel dataset and benchmarks for urban scene understanding in 2D and 3D. 45(3):3292–3310, 2023. 1, 2, 4

[15] Chieh Hubert Lin, Hsin-Ying Lee, Willi Menapace, Menglei Chai, Aliaksandr Siarohin, Ming-Hsuan Yang, and Sergey Tulyakov. InfiniCity: Infinite-scale city synthesis. 2023. 1

[16] Fan Lu, Kwan-Yee Lin, Yan Xu, Hongsheng Li, Guang Chen, and Changjun Jiang. Urban Architect: Steerable 3D urban scene generation with layout prior. *arXiv*, 2404.06780, 2024. 2

[17] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. 2020. 1, 2

[18] Michael Niemeyer and Andreas Geiger. GIRAFFE: representing scenes as compositional generative neural feature fields. 2021. 2, 5

[19] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *NIPS*, 2017. 2

[20] René Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. 44(3):1623–1637, 2022. 4

[21] Yuan Shen, Wei-Chiu Ma, and Shenlong Wang. SGAM: building a virtual 3D world through simultaneous generation and mapping. 2022. 2, 4, 5

[22] Xiaoyang Wu, Li Jiang, Peng-Shuai Wang, Zhijian Liu, Xihui Liu, Yu Qiao, Wanli Ouyang, Tong He, and Hengshuang Zhao. Point transformer V3: simpler, faster, stronger. 2024. 1, 2, 3

[23] Haozhe Xie, Zhaoxi Chen, Fangzhou Hong, and Ziwei Liu. CityDreamer: Compositional generative model of unbounded 3D cities. 2024. 1, 2, 4, 5

[24] Haozhe Xie, Zhaoxi Chen, Fangzhou Hong, and Ziwei Liu. CityDreamer4D: Compositional generative model of unbounded 4D cities. *arXiv*, 2501.08983, 2025. 2

[25] Yuanbo Yang, Yifei Yang, Hanlei Guo, Rong Xiong, Yue Wang, and Yiyi Liao. UrbanGIRAFFE: Representing urban scenes as compositional generative neural feature fields. 2023. 2, 4, 5

AI-Generated